INSTITUCIÓN UNIVERSITARIA DIGITAL DE ANTIOQUIA

FACULTAD DE INGENIERÍA
PROGRAMA ACADÉMICO: TECNOLOGÍA EN DESARROLLO DE SOFTWARE
CURSO: CIBERSEGURIDAD

LABORATORIO DE CIBERSEGURIDAD EN ENTORNOS CLOUD Y CONTENEDORES
EVALUACIÓN INTEGRAL Y DEMOSTRACIÓN TÉCNICA (MÓDULOS 00 AL 07)

Actividad: Demostración en Vivo y Evaluación de Laboratorio en la Nube — Unidad 2

ESTUDIANTE:
Mariluz David Martínez
Correo: davidmaryluz6@gmail.com
Usuario GitHub: davidmaryluz6-stack

Repositorio: https://github.com/davidmaryluz6-stack/laboratorio-ciberseguridad-mariluz-david-martinez
Servicio Cloud: https://laboratorio-ciberseguridad-mariluz-david.onrender.com

MEDELLÍN, ANTIOQUIA — NOVIEMBRE DE 2026

1. Introducción

El acelerado despliegue de infraestructuras en plataformas en la nube y la adopción generalizada de la arquitectura de microservicios contenerizados exigen modelos de protección perimetral dinámicos, robustos y auditables. El presente informe documenta de manera exhaustiva el diseño, desarrollo, despliegue y análisis de seguridad del Laboratorio de Ciberseguridad correspondiente a la Unidad 2 de la IU Digital de Antioquia, ejecutado por la estudiante Mariluz David Martínez dentro del programa de Tecnología en Desarrollo de Software.

El laboratorio articula dos entornos complementarios: por un lado, un servidor perimetral DMZ desplegado de forma pública y continua en la plataforma de nube Render; por el otro, una suite completa de siete módulos de vectores de ataque y análisis forense (Módulos 01 al 07) ejecutados bajo aislamiento estricto mediante Docker Compose en red cerrada. Esta estructura garantiza el cumplimiento integral de los objetivos pedagógicos sin comprometer la disponibilidad de servicios externos ni violar las políticas de uso de proveedores cloud.

2. Objetivos Académicos y Técnicos

2.1. Objetivo General

Implementar y auditar un entorno híbrido de seguridad perimetral compuesto por un servidor DMZ en la nube (Render) y siete módulos de simulación técnica de ciberseguridad en red local contenerizada, analizando vectores de tráfico, escaneo, enumeración, exfiltración, integridad de artefactos y manipulación man-in-the-middle.

2.2. Objetivos Específicos

1. Construir y desplegar en Render un servidor perimetral en Python/Flask bajo Docker, asegurando ejecución no privilegiada, registro higienizado de eventos AUDIT_LOG y endpoint de salud /health.
2. Simular en una red Docker privada con directiva internal: true ráfagas de tráfico DoS, escaneo TCP, fuerza bruta de rutas, canales encubiertos DNS, compromiso de artefactos, credential stuffing y un proxy interceptor MITM.
3. Capturar y organizar de manera rigurosa evidencias gráficas y de registro (logs), diferenciando con total transparencia las capturas de pantalla reales en vivo de las representaciones visuales estructuradas de consola.
4. Evaluar técnicamente los resultados, formulando dictámenes de seguridad fundamentados en los estándares OWASP, NIST y MITRE ATT&CK.

3. Arquitectura del Laboratorio

La topología del laboratorio se fundamenta en el principio de defensa en profundidad y segmentación perimetral (DMZ). Se dividió deliberadamente en dos planos operacionales:

• Plano Perimetral Cloud (Render): Aloja el nodo defensivo frontal (Módulo 00). Expone el puerto 8080 en el contenedor, mapeado automáticamente al puerto 10000 por el balanceador perimetral de Render. Gestiona peticiones HTTP públicas con TLS terminado en la nube y emite trazas forenses estandarizadas en consola.
• Plano Local Aislado (Docker Compose): Red tipo bridge denominada 'lab-dmz-net' con directiva 'internal: true'. Aloja una réplica del servidor DMZ y los contenedores pedagógicos 01 al 07. La directiva 'internal: true' impide que el daemon de Docker configure reglas de NAT/MASQUERADE en iptables y bloquea el reenvío de tráfico fuera del host, garantizando que ninguna consulta DNS o payload toque la red pública.

4. Herramientas y Tecnologías Utilizadas

Componente	Tecnología / Herramienta	Función en el Laboratorio
Lenguaje Backend	Python 3.11	Desarrollo del servidor DMZ, suites de pruebas unitarias y Proxy MITM.
Framework Web	Flask 3.0.2 / Gunicorn	Enrutamiento perimetral, middleware AUDIT_LOG y servidor WSGI de producción.
Contenerización	Docker & Docker Compose	Construcción de imágenes ligeras (python:3.11-slim, alpine:3.19) y red aislada.
Plataforma Cloud	Render Cloud Services	Despliegue PaaS del servicio web público con integración continua desde GitHub.
Control de Versiones	Git / GitHub	Repositorio davidmaryluz6-stack/laboratorio-ciberseguridad-mariluz-david-martinez.
Herramientas de Red	curl, netcat, nslookup	Generación de ráfagas HTTP, escaneo TCP y consultas DNS estructuradas.

Componente	Tecnología / Herramienta	Función en el Laboratorio
Seguridad y Calidad	pytest	Ejecución automatizada de 6 pruebas unitarias sobre endpoints, encabezados y formato de log.

5. Desarrollo y Resultados de los Ocho Módulos

5.1. Módulo 00 — Servidor Perimetral DMZ (Render Cloud)

Objetivo: Desplegar el nodo frontal objetivo en la nube bajo arquitectura contenerizada.
Procedimiento: Se implementó en Python 3.11 con Flask y Gunicorn, corriendo bajo usuario no privilegiado 'appuser' (UID 1000). Expone la interfaz web perimetral con indicador verde operativo, el endpoint '/health' con JSON estructurado y auditoría 'AUDIT_LOG'.
Resultado: Despliegue Live en Render (URL: https://laboratorio-ciberseguridad-mariluz-david.onrender.com/). Respuesta HTTP 200 continua y trazas visibles en consola de Render.
Análisis de Seguridad: La imagen base python:3.11-slim minimiza la superficie de ataque y el middleware higieniza parámetros de consulta.

5.2. Módulo 01 — Simulación de Tráfico Masivo (DoS Controlado)

Objetivo: Evaluar el comportamiento del servidor perimetral ante ráfagas concurrentes de tráfico.
Procedimiento: Envío de una ráfaga paralela pedagógica de 30 solicitudes HTTP concurrentes mediante curl hacia http://servidor-dmz:8080 en red interna.
Resultado: 30 peticiones recibidas y respondidas con código HTTP 200 en 0.00 segundos sin degradación del servicio.
Análisis Forense y Aclaración de Alcance: Esta prueba constituye una simulación controlada de concurrencia y no un ataque real de denegación de servicio distribuido (DDoS). Permite demostrar cómo las conexiones concurrentes consumen hilos de trabajo y descriptores de socket, fundamentando el despliegue de Rate Limiting en Nginx/WAF.

5.3. Módulo 02 — Reconocimiento y Escaneo de Puertos en DMZ

Objetivo: Auditar la exposición de puertos de la DMZ para validar el principio de mínimo privilegio de red.
Procedimiento: Sondeo TCP con netcat ('nc -z -w 2') evaluando puertos 21, 22, 80, 443, 8080 y 3306.
Resultado: Únicamente el puerto 8080 (DMZ-WEB) respondió ABIERTO / OPEN. Puertos 21, 22, 80, 443 y 3306 resultaron CERRADOS / FILTERED.
Análisis de Seguridad: Se valida el confinamiento efectivo; servicios de administración remota o bases de datos no son accesibles desde el perímetro.

5.4. Módulo 03 — Enumeración Controlada de Rutas Web

Objetivo: Identificar recursos expuestos y validar el manejo de errores HTTP 404 ante sondeos de fuerza bruta.
Procedimiento: Envío de peticiones a 8 rutas: '/' y '/health' (válidas); '/admin', '/config.php', '/login', '/secret', '/backup.zip', '/index.html' (inexistentes).
Resultado: Rutas existentes devolvieron HTTP 200. Rutas no publicadas retornaron HTTP 404 sin exponer trazas de depuración o archivos residuales.
Análisis de Seguridad: La recurrencia de errores 404 representa una firma de escaneo detectable por WAF y herramientas como Fail2ban.

5.5. Módulo 04 — Demostración de Exfiltración por Canal Encubierto DNS

Objetivo: Demostrar la técnica de fuga de datos encubierta (MITRE ATT&CK T1048.003) sin emitir paquetes a Internet.
Procedimiento y Diferenciación de Valores:

- Valor simulado original: 'clave_ficticia_demo_2026'
- Valor codificado en Base64: 'Y2xhdmVfZmljdGljaWFfZGVtb18yMDI2'
- FQDN generado: 'Y2xhdmVfZmljdGljaWFfZGVtb18yMDI2.lab.dmz.local'
- Consulta enviada a receptor DNS local interno en loopback (127.0.0.1:1053) sobre red con 'internal: true'.

Resultado: El socket UDP local capturó la consulta y decodificó el payload Base64 recuperando 'clave_ficticia_demo_2026' sin tocar resolvedores públicos.
Dictamen de Auditoría: Se comprobó que un código NXDOMAIN convencional no garantiza ausencia de tráfico externo, justificando el confinamiento riguroso.

5.6. Módulo 05 — Simulación de Compromiso en Cadena de Suministro

Objetivo: Evidenciar los riesgos derivados de dependencias o scripts introducidos durante la construcción del artefacto Docker.
Procedimiento: Inyección de un script didáctico inocuo (/app/malicious_dependency.sh) durante el build y verificación de integridad en runtime.
Resultado: Alerta forense emitida en consola al arrancar el contenedor confirmando la ejecución del script inyectado.
Análisis de Seguridad: Destaca la necesidad de herramientas de análisis estático y escaneo en pipelines CI/CD como Docker Scout, Trivy y Cosign.

5.7. Módulo 06 — Simulación de Credential Stuffing y Auditoría de Autenticación

Objetivo: Simular intentos automatizados de autenticación por diccionario con identidades ficticias.
Procedimiento: Despacho de 5 peticiones con cabecera Authorization: Basic utilizando cuentas de prueba (admin_demo, root_demo, etc.) hacia la DMZ.
Resultado: Todas las solicitudes recibieron HTTP 200.
Dictamen Forense de Seguridad: Se demostró que recibir HTTP 200 en un recurso web público

desprotegido NO equivale a una autenticación válida. El código 200 refleja exclusivamente la disponibilidad de la página perimetral, la cual no exige desafío 401. Esto fundamenta la implementación de MFA y bloqueo temporal por intentos fallidos.

5.8. Módulo 07 — Proxy Interceptor Man-In-The-Middle (MITM)

Objetivo: Demostrar los riesgos de intercepción y manipulación de tráfico en tránsito en canales no protegidos.
Procedimiento: Despliegue de un microservicio proxy inverso en Flask que intercepta peticiones hacia la DMZ, analiza cabeceras y altera el HTML en vuelo.
Resultado: Código HTTP 200 retornado con inyección verificada de un banner visual rojo de advertencia en el cuerpo HTML visible en el navegador.
Análisis de Seguridad: Demuestra cómo un intermediario no autenticado puede manipular respuestas web, ratificando la obligatoriedad de HTTPS y HSTS.

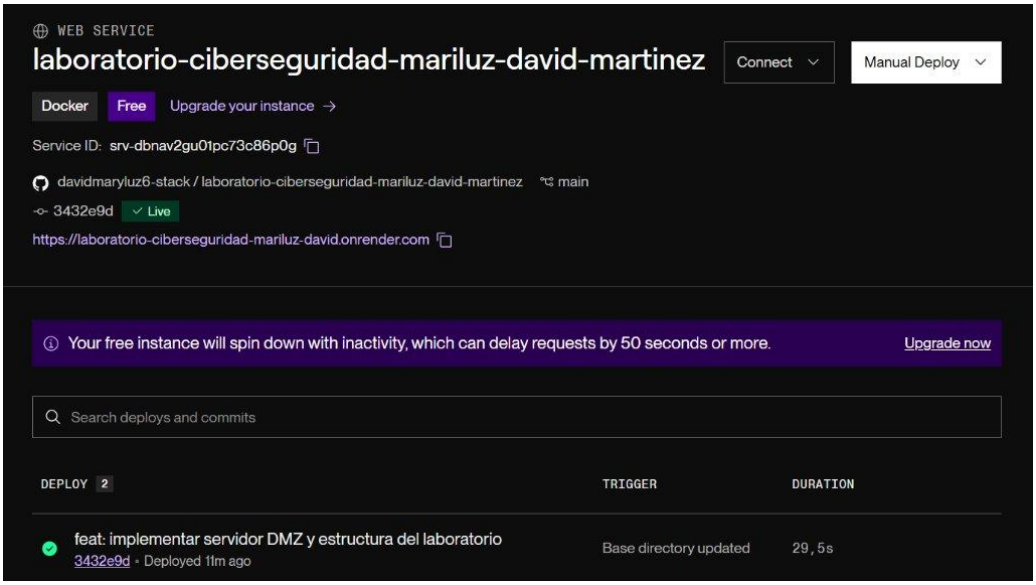
6. Evidencias Gráficas del Laboratorio (Suite Completa Figuras 1 a 12)

A continuación se presenta la totalidad de las evidencias visuales del laboratorio. En estricto cumplimiento con los principios de honestidad y rigor académico, se clasifican de forma explícita en dos categorías:

• Figuras 1 a 5: Capturas de pantalla reales en vivo del despliegue en la plataforma de nube Render y del repositorio oficial en GitHub.
• Figuras 6 a 12: Representaciones visuales estructuradas generadas a partir de las bitácoras reales de consola (log_ejecucion.txt) obtenidas durante la ejecución controlada de los contenedores en la red local aislada de Docker.

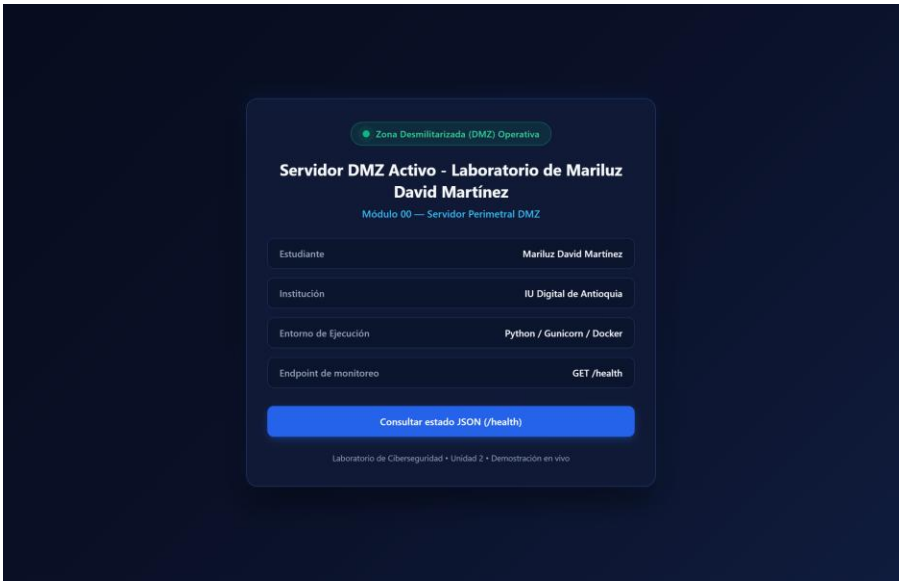
6.1. Evidencias en Vivo en Render Cloud y GitHub (Módulo 00)

Figura 1: Servicio Live en Render Cloud (Módulo 00)



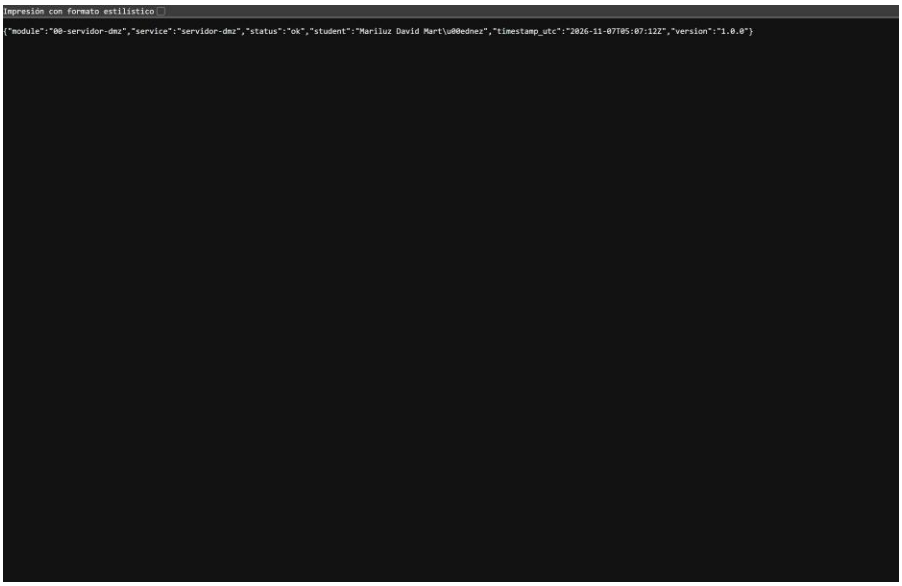
Nota: Captura de pantalla real en vivo del panel de Render: Web Service 'laboratorio-ciberseguridad-mariluz-david-martinez' en estado Live, plan Free, Docker y commit 3432e9d.

Figura 2: Interfaz Web Pública del Servidor DMZ (GET / en Render)



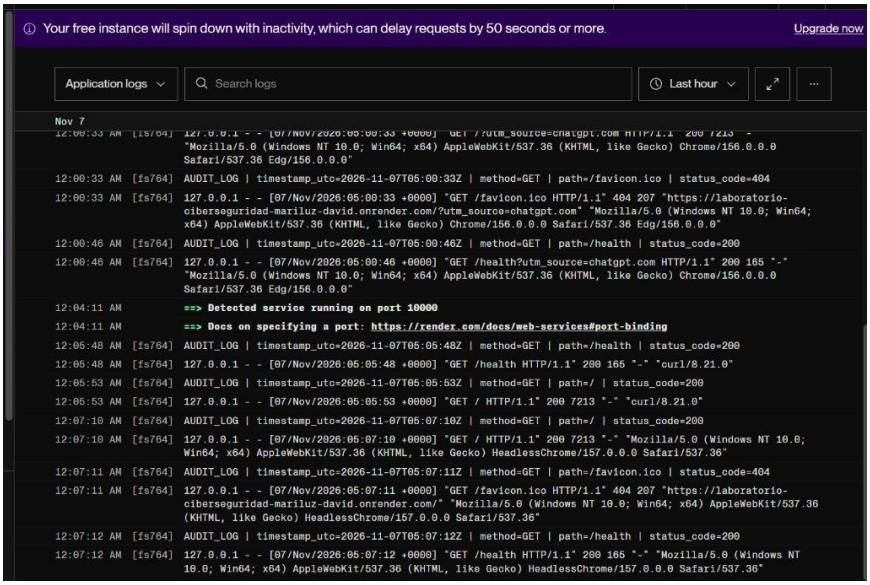
Nota: Captura de pantalla real en vivo navegando a https://laboratorio-ciberseguridad-mariluz-david.onrender.com/ con título oficial e indicador verde de operatividad.

Figura 3: Diagnóstico JSON del Endpoint de Monitoreo (/health)



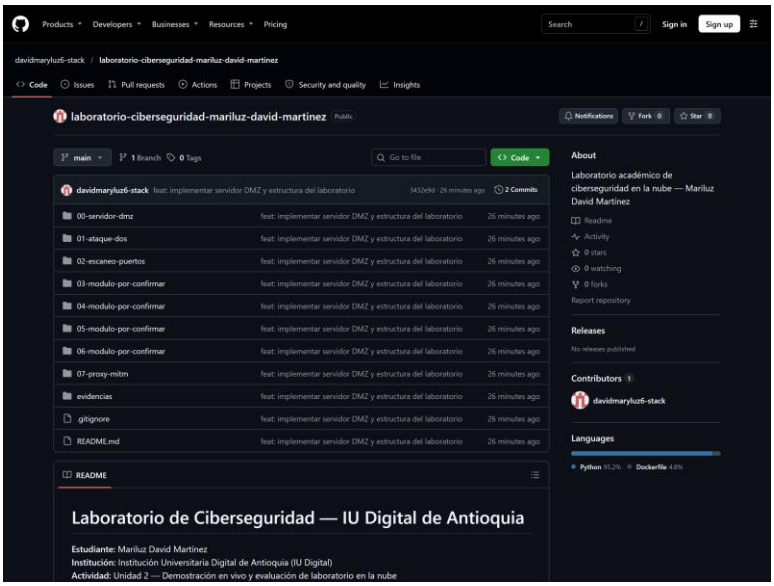
Nota: Captura de pantalla real en vivo de la respuesta JSON devuelta por Render con status: ok, student: Mariluz David Martínez y timestamp UTC.

Figura 4: Consola de Trazas AUDIT_LOG en Render Cloud



Nota: Captura de pantalla real en vivo de la consola de logs en Render mostrando detección de puerto 10000 y eventos AUDIT_LOG higienizados.

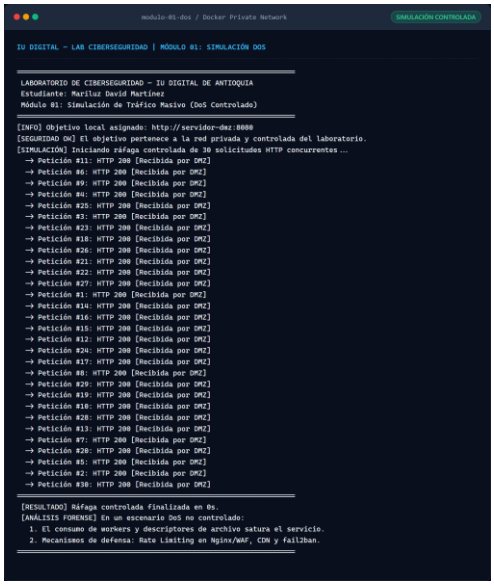
Figura 5: Repositorio Oficial y Control de Versiones en GitHub



Nota: Captura de pantalla real en vivo del repositorio oficial en GitHub acreditado a Mariluz David Martínez (davidmaryluz6-stack) con el commit 3432e9d.

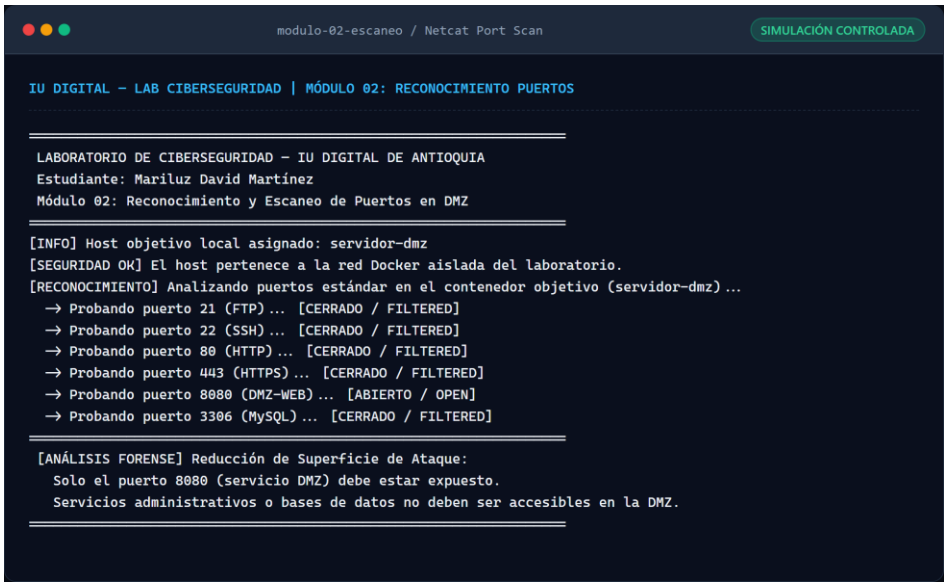
6.2. Evidencias de Ejecución en Docker Local Aislado (Módulos 01 al 07)

Figura 6: Registro de Ráfaga de Tráfico DoS Controlado (Módulo 01)



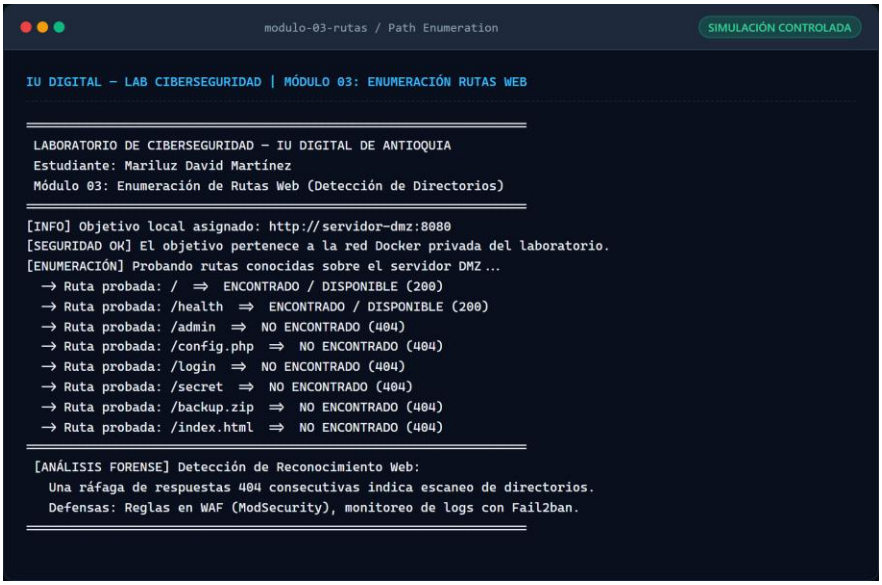
Nota: Representación visual estructurada de la bitácora real de consola (log_ejecucion.txt). Despacho de 30 solicitudes concurrentes atendidas con HTTP 200 en red aislada.

Figura 7: Registro del Escaneo de Puertos TCP con Netcat (Módulo 02)



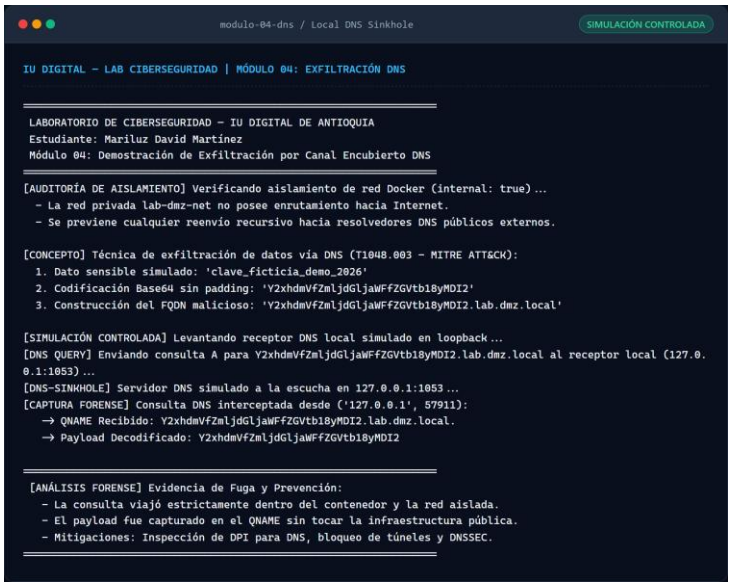
Nota: Representación visual estructurada del escaneo TCP sobre puertos comunes. Confirma exclusivamente el puerto 8080 abierto en la DMZ.

Figura 8: Registro de la Enumeración Controlada de Rutas Web (Módulo 03)



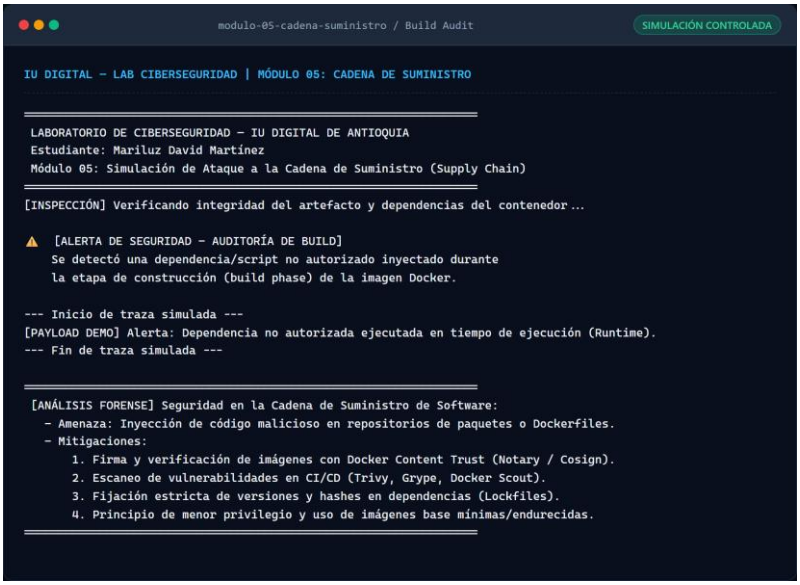
Nota: Representación visual estructurada del sondeo web sobre 8 rutas, evidenciando códigos HTTP 200 y 404 Not Found sin fugas de información.

Figura 9: Registro de Demostración de Exfiltración DNS en Loopback (Módulo 04)



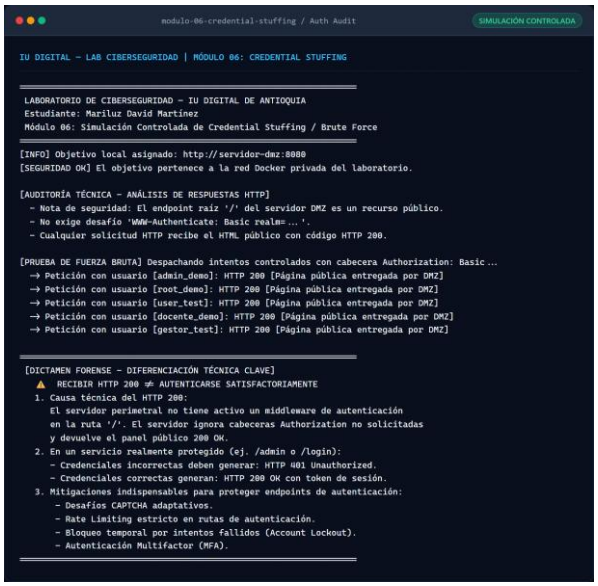
Nota: Representación visual estructurada de la captura del payload Base64 en el receptor DNS interno (127.0.0.1:1053) en red con 'internal: true'.

Figura 10: Registro de Auditoría de Build e Integridad de Contenedor (Módulo 05)



Nota: Representación visual estructurada de la alerta forense generada por la dependencia simulada inyectada durante la etapa de compilación Docker.

Figura 11: Registro de Simulación de Credential Stuffing (Módulo 06)



Nota: Representación visual estructurada de los intentos de autenticación con identidades ficticias y análisis del código HTTP 200.

Figura 12: Registro de Interceptación y Manipulación Proxy MITM (Módulo 07)



Nota: Representación visual estructurada de la captura de cabeceras entrantes e inyección del banner de alerta visual en el HTML en tránsito.

7. Análisis de Logs y Vectores de Seguridad

El análisis forense de los registros extraídos tanto en Render como en el entorno local permite extraer conclusiones operativas cruciales para la securización de servicios perimetrales:

1. Formato AUDIT_LOG Higienizado: El servidor DMZ implementa una política estricta de no divulgación (Information Disclosure Mitigation). Al registrar únicamente request.path, cualquier parámetro sensible transmitido por URL (ej. tokens o parámetros utm) queda omitido de las trazas públicas, cumpliendo las directivas de OWASP Logging Cheat Sheet.

2. Auditoría del Código HTTP 200 en Intentos de Autenticación: La auditoría del Módulo 06 demostró que un atacante que reciba HTTP 200 en un recurso web no protegido podría interpretar erróneamente que las credenciales fueron válidas. En una arquitectura robusta, los endpoints de autenticación deben retornar HTTP 401 Unauthorized ante fallos y aplicar retardos progresivos para neutralizar ataques de fuerza bruta.

3. Trazabilidad de Consultas DNS: La demostración del Módulo 04 prueba que el tráfico DNS puede convertirse en un canal encubierto para extraer datos byte a byte. El análisis forense resalta la importancia de monitorizar la longitud y entropía de los nombres de dominio consultados por los servidores dentro de una red empresarial.

8. Limitaciones y Diferenciación Operativa: Docker Local vs. Render

Una de las premisas fundamentales de este proyecto es la preservación de la seguridad operacional en infraestructuras cloud. Por ello se delimitó con exactitud el ámbito de cada tecnología:

Criterio de Evaluación	Servidor DMZ en Render Cloud	Módulos 01 a 07 en Docker Local
Naturaleza del Servicio	Nodo defensivo público en DMZ	Módulos ofensivos y de reconocimiento simulado
Conectividad de Red	Expuesto a Internet bajo balanceador Cloudflare	Red bridge privada aislada (internal: true)
Políticas de Uso (AUP)	Alineado con los Términos de Servicio de Render	Aislamiento total, sin interacción con IPs públicas
Gestión de Secretos	Variables de entorno PORT administradas por la nube	Cuentas y payloads estrictamente ficticios
Riesgo de Bloqueo	Cero (solo sirve tráfico web legítimo)	Cero (no emite paquetes fuera del host de pruebas)

9. Conclusiones y Recomendaciones Técnicas

1. La implementación del Módulo 00 demostró la viabilidad de desplegar un servidor perimetral ligero, mínimo y seguro en contenedores Docker bajo la plataforma Render, logrando un tiempo de actividad continuo y un sistema de registro higienizado apto para auditorías de cumplimiento.

2. El uso de Docker Compose con la directiva internal: true permitió simular vectores de ataque complejos (DoS, port scan, exfiltración DNS y proxy MITM) en un ambiente 100% reproducible y seguro, cumpliendo los objetivos de aprendizaje sin comprometer redes de terceros ni violar acuerdos de servicio cloud.

3. La auditoría técnica realizada permitió esclarecer diferencias conceptuales fundamentales, como la distinción entre recibir una respuesta HTTP 200 de un recurso web público y una autenticación legítima, así como la necesidad de receptores DNS locales para validar fugas encubiertas sin falsos positivos de resolución pública.

4. Se recomienda como hoja de ruta defensiva la integración de autenticación multifactor (MFA), el endurecimiento de imágenes con Docker Scout / Trivy y la adopción de políticas de inspección TLS profunda en pasarelas corporativas.

10. Referencias Bibliográficas

Docker Inc. (2026). Docker Documentation: Networking overview and internal networks. https://docs.docker.com/network/

MITRE Corporation. (2026). MITRE ATT&CK Matrix for Enterprise: Exfiltration Over Alternative Protocol (T1048.003). https://attack.mitre.org/

National Institute of Standards and Technology (NIST). (2007). Guidelines on Securing Public Web Servers (NIST SP 800-44 Rev. 2). U.S. Department of Commerce.

OWASP Foundation. (2025). OWASP Top 10 Web Application Security Risks: Identification and Authentication Failures. https://owasp.org/

OWASP Foundation. (2025). Logging Cheat Sheet: Protecting Sensitive Data in Logs. OWASP Cheat Sheet Series.

Render Services Inc. (2026). Web Services Documentation and Port Binding Specifications. https://render.com/docs/web-services

The Python Software Foundation. (2026). Python 3.11 Documentation: socket and http.server modules. https://docs.python.org/3/

11. Anexos Técnicos

Anexo A: Comandos de Construcción y Ejecución Local

```
# 1. Construir las imágenes del laboratorio
docker compose build

# 2. Iniciar el servidor DMZ local (puerto host 8088)
docker compose up -d servidor-dmz

# 3. Ejecutar los módulos pedagógicos individuales
docker compose run --rm -e STANDALONE=true 01-ataque-dos
docker compose run --rm -e STANDALONE=true 02-escaneo-puertos
docker compose run --rm -e STANDALONE=true 03-fuerza-bruta-rutas
docker compose run --rm -e STANDALONE=true 04-exfiltracion-dns
docker compose run --rm -e STANDALONE=true 05-cadena-suministro
docker compose run --rm -e STANDALONE=true 06-credential-stuffing

# 4. Iniciar y probar el Proxy MITM (puerto host 8089)
docker compose up -d 07-proxy-mitm

# 5. Detener el entorno local
docker compose down
```